

■ Ollama Crash Course

Intern Training Guide — Local LLMs Made Simple

1. What is Ollama?

Ollama is a free, open-source tool that lets you run large language models (LLMs) **entirely on your own machine** — no cloud, no API key, no internet required once the model is downloaded.

Why Ollama?

- ■ **Privacy** — your data never leaves your machine
- ■ **Free** — no usage costs or rate limits
- ■ **Fast** — models run locally on CPU or GPU
- ■ **Simple API** — works like OpenAI's API, so migration is easy

Supported Models

- **llama3.2** — Meta's fast, capable model (recommended for beginners)
- **mistral** — Mistral AI's efficient 7B model
- **gemma2** — Google's lightweight model
- **phi3** — Microsoft's small but surprisingly powerful model

■ **Tip:** All models are pulled from the Ollama model library at ollama.com/library

2. Installation

Install Ollama

- **macOS:**

```
brew install ollama
```

- **Linux:**

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **Windows:** Download the installer from ollama.com and run it.

Start the Server

Ollama runs as a local server on **port 11434**. Start it with:

```
ollama serve
```

■ **Tip:** On macOS/Linux, Ollama may auto-start after install. You can check by visiting <http://localhost:11434> in your browser.

Pull a Model

Download a model to your machine (one-time, like docker pull):

```
ollama pull llama3.2
```

List all downloaded models:

```
ollama list
```

3. Simple LLM Call (Python)

Install the Python Client

```
pip install ollama
```

Minimal Example

Send a single message to the model and print the reply:

```
import ollama

response = ollama.chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "What is the capital of France?"}]
)

print(response["message"]["content"])
# Output: Paris is the capital of France.
```

Response Structure

The response object contains:

- `response["message"]["content"]` — the model's text reply
- `response["model"]` — which model responded
- `response["done"]` — True when generation is complete

4. Stateless vs Stateful (Memory) Calls

Stateless Calls

Each call is **completely independent**. The model has no memory of previous messages. Use this for single-turn Q&A.;

```
# Turn 1 – model has no context
ollama.chat(model='llama3.2',
            messages=[{'role': 'user', 'content': 'My name is Ahmed.'}])

# Turn 2 – model has FORGOTTEN Turn 1
ollama.chat(model='llama3.2',
            messages=[{'role': 'user', 'content': 'What is my name?'}])
# Output: I don't know your name.
```

Stateful Calls (Simulated Memory)

LLMs are stateless by design. To simulate memory, **accumulate all messages** in a list and pass the full history on every call:

```
messages = [] # grows with each turn

def chat_with_memory(user_input):
    messages.append({'role': 'user', 'content': user_input})
    response = ollama.chat(model='llama3.2', messages=messages)
    reply = response['message']['content']
    messages.append({'role': 'assistant', 'content': reply})
    return reply

chat_with_memory('My name is Mohamed.') # stored in list
chat_with_memory('What is my name?')   # model sees full history
# Output: Your name is Mohamed.
```

■ **Tip:** The messages list is the 'memory'. Longer history = more context tokens used.

5. Tool Calling

What Are Tools?

Tools (also called **function calling**) allow the LLM to **request that your code runs a function** on its behalf. Instead of hallucinating an answer, the model can invoke a real Python function and use the result.

The Tool Calling Loop

- 1. You send a prompt + a list of tool definitions (JSON schemas)
- 2. The model replies with a **tool_call** instead of plain text
- 3. Your code executes the requested function
- 4. You send the result back to the model
- 5. The model gives a final natural-language response

Example: sum_two_numbers

Step 1 — Define the tool schema (tells the model what functions exist):

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'sum_two_numbers',
        'description': 'Returns the sum of two numbers',
        'parameters': {
            'type': 'object',
            'properties': {
                'a': {'type': 'number', 'description': 'First number'},
                'b': {'type': 'number', 'description': 'Second number'}
            },
            'required': ['a', 'b']
        }
    }
}]
```

Step 2 — Implement the actual Python function:

```
def sum_two_numbers(a, b):
    return a + b
```

Step 3 & 4 — Send request, handle tool call, return result:

```
import json

response = ollama.chat(
    model='llama3.2',
    messages=[{'role': 'user', 'content': 'What is 7 plus 15?'}],
    tools=tools
)

message = response['message']
if message.get('tool_calls'):
    for tool_call in message['tool_calls']:
        fn_name = tool_call['function']['name']
        args = tool_call['function']['arguments']
        if fn_name == 'sum_two_numbers':
            result = sum_two_numbers(**args) # execute the function
            print(f'Tool result: {result}') # 22

# Step 5: send result back to model
final = ollama.chat(
    model='llama3.2',
    messages=[
        {'role': 'user', 'content': 'What is 7 plus 15?'},
        {'role': 'assistant', 'content': None,
         'tool_calls': message['tool_calls']},
        {'role': 'tool', 'content': str(result)}
    ]
)
print(final['message']['content'])
```

```
# Output: 7 plus 15 equals 22.
```

6. Quick Reference Table

Concept	What It Means	Key Code Pattern
Stateless call	Single-turn, no memory	<code>ollama.chat(model, messages=[one_msg])</code>
Stateful call	Multi-turn with memory via message list	<code>ollama.chat(model, messages=history)</code>
Tool definition	JSON schema describing a callable function	<code>tools=[{"type": "function", "function": {...}}]</code>
Tool call detection	Model requests a function instead of answering	<code>response["message"].get("tool_calls")</code>
Tool result	Send function output back so model can answer	<code>{"role": "tool", "content": str(result)}</code>
Pull model	Download a model once (like docker pull)	<code>ollama pull llama3.2</code>
List models	See all downloaded models	<code>ollama list</code>
Start server	Launch the local Ollama API server	<code>ollama serve</code>

For full documentation visit ollama.com · Model library at ollama.com/library · Python client docs: github.com/ollama/ollama-python